

Se complemento la solución y la generación de reportes de la automatización para la generación de reportes de trámites, mediante la construcción de un script en Python que permite procesar de manera sistemática la información consolidada y generar reportes individuales por municipio, a partir de una plantilla (TuCatastro) estructurada previamente definida.

Documentación Código y Reportes:

https://github.com/cientificoacc/2026/tree/main/Reportes_Automatizacion_ACC

Codigo:

```
# LIBRERÍAS
!pip install -q python-docx reportlab matplotlib

import pandas as pd
import os
import matplotlib.pyplot as plt
from docx import Document
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Image
as RLImage
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.units import inch
from reportlab.lib.pagesizes import letter
import locale

try:
    locale.setlocale(locale.LC_ALL, 'en_US.UTF-8')
except:
    locale.setlocale(locale.LC_ALL, '')

# 2. CONFIGURACIÓN GENERAL

BASE_EXCEL = "/content/Plantilla_Reporte_TuCatastro.xlsx"
PLANTILLA_WORD = "/content/Mpio_Informe_tramites_Catastral_ACC.docx"

ANIOS = [2024, 2025]
CARPETA_RAIZ = "/content/Reportes_ACC_2024_2025"

# 3. NORMALIZACIÓN DE COLUMNAS

def normalizar_columnas(df):
    df.columns = df.columns.str.strip()
```

```

df.columns = (
    df.columns
    .str.normalize('NFKD')
    .str.encode('ascii', errors='ignore')
    .str.decode('utf-8')
)
return df

```

4. DETECCIÓN AUTOMÁTICA DE COLUMNA FECHA

```

def detectar_columna_fecha(df):
    posibles = [c for c in df.columns if 'fecha' in c.lower() and 'rad' in
c.lower()]
    if not posibles:
        raise ValueError("✗ No se encontró columna de fecha de radicación
en el Excel.")
    return posibles[0]

```

5. FUNCIONES DE TABLAS

```

def construir_tabla1(df):

    t = (
        df.groupby(['Estado', 'RAD_ANO'])['Numero radicado']
        .nunique()
        .unstack(fill_value=0)
        .reindex(columns=[2024, 2025], fill_value=0)
    )

    # Total horizontal
    t['Total general'] = t.sum(axis=1)

    # Total vertical
    t.loc['Total general'] = t.sum()

    t = (
        t.reset_index()
        .rename(columns={
            'Estado': 'Estado de la radicación',
            2024: '2024',
            2025: '2025'
        })
    )

```

```

    )

    return t

def construir_tabla2(df):

    base = (
        df.groupby(['OFICINA DE GESTION', 'Tipo', 'RAD_ANO'])['Numero
radicado']
        .nunique()
        .unstack(fill_value=0)
        .reindex(columns=[2024, 2025], fill_value=0)
    )

    base['Total general'] = base.sum(axis=1)
    base = base.reset_index()

    base.columns = [
        'Oficina de Gestión',
        'Tipo de trámite',
        '2024',
        '2025',
        'Total general'
    ]

    base['Nivel'] = base['Oficina de Gestión'].apply(
        lambda x: 'Central' if 'CENTRAL' in str(x).upper() else
'Territorio'
    )

    filas_finales = []

    for nivel in ['Central', 'Territorio']:

        bloque = base[base['Nivel'] == nivel]

        for _, row in bloque.iterrows():
            filas_finales.append(row.drop('Nivel').to_dict())

    subtotal = {
        'Oficina de Gestión': f'Subtotal {nivel}',
        'Tipo de trámite': '',
        '2024': bloque['2024'].sum(),
        '2025': bloque['2025'].sum(),
        'Total general': bloque['Total general'].sum()
    }

```

```

    }

    filas_finales.append(subtotal)

total_general = {
    'Oficina de Gestión': 'Total general',
    'Tipo de trámite': '',
    '2024': base['2024'].sum(),
    '2025': base['2025'].sum(),
    'Total general': base['Total general'].sum()
}

filas_finales.append(total_general)

return pd.DataFrame(filas_finales)

def construir_tabla3(df):

    df = df.copy()

    df['RES_ANO_GROUP'] = 'En proceso'
    df.loc[df['RES_ANO'] == 2024, 'RES_ANO_GROUP'] = 'Resolución 2024'
    df.loc[df['RES_ANO'] == 2025, 'RES_ANO_GROUP'] = 'Resolución 2025'

    base = (
        df.groupby(['RAD_ANO', 'OFICINA DE
GESTION', 'Tipo', 'RES_ANO_GROUP'])
            ['Numero radicado']
            .nunique()
            .unstack(fill_value=0)
            .reindex(columns=['En proceso', 'Resolución 2024', 'Resolución
2025'], fill_value=0)
    )

    base['Total general'] = base.sum(axis=1)
    base = base.reset_index()

    base.columns = [
        'Año',
        'Oficina de Gestión',
        'Tipo de trámite',
        'En proceso',
        'Resolución 2024',
        'Resolución 2025',
        'Total general'
    ]

```

```

]

base['Nivel'] = base['Oficina de Gestión'].apply(
    lambda x: 'Central' if 'CENTRAL' in str(x).upper() else
'Territorio'
)

filas_finales = []

for anio in [2024, 2025]:

    bloque_anio = base[base['Año'] == anio]

    for nivel in ['Central', 'Territorio']:

        bloque = bloque_anio[bloque_anio['Nivel'] == nivel]

        for _, row in bloque.iterrows():
            filas_finales.append(row.drop('Nivel').to_dict())

        subtotal = {
            'Año': anio,
            'Oficina de Gestión': f'Subtotal {nivel}',
            'Tipo de trámite': '',
            'En proceso': bloque['En proceso'].sum(),
            'Resolución 2024': bloque['Resolución 2024'].sum(),
            'Resolución 2025': bloque['Resolución 2025'].sum(),
            'Total general': bloque['Total general'].sum()
        }

        filas_finales.append(subtotal)

total_general = {
    'Año': 'Total general',
    'Oficina de Gestión': '',
    'Tipo de trámite': '',
    'En proceso': base['En proceso'].sum(),
    'Resolución 2024': base['Resolución 2024'].sum(),
    'Resolución 2025': base['Resolución 2025'].sum(),
    'Total general': base['Total general'].sum()
}

filas_finales.append(total_general)

return pd.DataFrame(filas_finales)

```

```
# 6. FUNCIONES DE GRÁFICAS
```

```
def graficar_mes(df, ruta):
```

```
    col_fecha = detectar_columna_fecha(df)
```

```
    df = df.copy()
```

```
    df[col_fecha] = pd.to_datetime(df[col_fecha], errors='coerce')
```

```
    # Crear columna numérica del mes
```

```
    df['MES_NUM'] = df[col_fecha].dt.month
```

```
    # Diccionario correcto (1-12)
```

```
    meses_es = {
```

```
        1: 'Enero',
```

```
        2: 'Febrero',
```

```
        3: 'Marzo',
```

```
        4: 'Abril',
```

```
        5: 'Mayo',
```

```
        6: 'Junio',
```

```
        7: 'Julio',
```

```
        8: 'Agosto',
```

```
        9: 'Septiembre',
```

```
        10: 'Octubre',
```

```
        11: 'Noviembre',
```

```
        12: 'Diciembre'
```

```
    }
```

```
    # Tabla base agrupada
```

```
    t = (
```

```
        df.groupby(['MES_NUM', 'RAD_ANO'])['Numero radicado']  
        .nunique()
```

```
        .unstack(fill_value=0)
```

```
        .reindex(columns=[2024, 2025], fill_value=0)
```

```
    )
```

```
    # Asegurar que estén los 12 meses
```

```
    t = t.reindex(range(1,13), fill_value=0)
```

```
    # Reemplazar índice numérico por nombre del mes
```

```
    t.index = t.index.map(meses_es)
```

```
    # Gráfico
```

```

ax = t.plot(kind='bar')

#plt.title("Radicados por Mes (2024 vs 2025)")
plt.xlabel("Mes")
plt.ylabel("Cantidad")
plt.legend(title="Año")
plt.xticks(rotation=45)

# Etiquetas automáticas
for container in ax.containers:
    ax.bar_label(container, fontsize=8)

max_val = t.values.max()
plt.ylim(0, max_val * 1.15 if max_val > 0 else 1)

plt.tight_layout()
plt.savefig(ruta)
plt.close()

def graficar_dia(df, ruta):

    col_fecha = detectar_columna_fecha(df)

    df = df.copy()
    df[col_fecha] = pd.to_datetime(df[col_fecha], errors='coerce')
    df['DIA_NUM'] = df[col_fecha].dt.weekday

    dias_es = {
        0: 'Lunes',
        1: 'Martes',
        2: 'Miércoles',
        3: 'Jueves',
        4: 'Viernes',
        5: 'Sábado',
        6: 'Domingo'
    }

    df['DIA'] = df['DIA_NUM'].map(dias_es)

    t = (
        df.groupby(['DIA_NUM', 'DIA', 'RAD_ANO'])['Numero radicado']
            .nunique()
            .unstack(fill_value=0)
            .reindex(columns=[2024, 2025], fill_value=0)
            .reset_index()
    )

```

```

        .sort_values('DIA_NUM')
        .set_index('DIA')
    )

    t = t[[2024, 2025]]

    ax = t.plot(kind='bar')

    #plt.title("Radicados por Día de la Semana (2024 vs 2025)")
    plt.xlabel("Día")
    plt.ylabel("Cantidad")
    plt.legend(title="Año")
    plt.xticks(rotation=45)

    # Etiquetas
    for container in ax.containers:
        ax.bar_label(container, fontsize=8)

    plt.ylim(0, t.values.max() * 1.15 if t.values.max() > 0 else 1)
    plt.tight_layout()
    plt.savefig(ruta)
    plt.close()

# REEMPLAZO REAL DE BLOQUES (PLANTILLA CON TEXTO PLANO)

from docx.shared import Inches

from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import RGBColor
from docx.enum.text import WD_ALIGN_PARAGRAPH

def formatear_tabla_word(tabla):

    azul = "2F5496"

    # -----
    # ENCABEZADOS
    # -----
    for cell in tabla.rows[0].cells:

        tc = cell._tc
        tcPr = tc.get_or_add_tcPr()

```

```

shd = OxmlElement('w:shd')
shd.set(qn('w:fill'), azul)
tcPr.append(shd)

for paragraph in cell.paragraphs:
    for run in paragraph.runs:
        run.font.bold = True
        run.font.color.rgb = RGBColor(255, 255, 255)

# -----
# SUBTOTALES Y TOTALES
# -----
for row in tabla.rows[1:]:

    texto_fila = row.cells[0].text.upper()

    if "SUBTOTAL" in texto_fila or "TOTAL GENERAL" in texto_fila:

        for cell in row.cells:

            # Fondo blanco (limpia cualquier shading previo)
            tc = cell._tc
            tcPr = tc.get_or_add_tcPr()

            shd = OxmlElement('w:shd')
            shd.set(qn('w:fill'), "FFFFFF")
            tcPr.append(shd)

            for paragraph in cell.paragraphs:
                for run in paragraph.runs:
                    run.font.bold = True
                    run.font.color.rgb = RGBColor(0, 0, 0)

def reemplazar_bloque(doc, titulo_busqueda, tipo, df=None,
ruta_imagen=None):

    paragraphs = doc.paragraphs
    inicio = None
    fin = None

    # -----
    # a. Encontrar párrafo del título
    # -----
    for i, p in enumerate(paragraphs):

```

```

        if titulo_busqueda.strip() in p.text.strip():
            inicio = i
            break

if inicio is None:
    print(f"⚠ No se encontró el título: {titulo_busqueda}")
    return

# -----
# b. Encontrar siguiente título (Tabla o Imagen)
# -----
for j in range(inicio + 1, len(paragraphs)):
    texto = paragraphs[j].text.strip()
    if texto.startswith("Tabla") or texto.startswith("Imagen"):
        fin = j
        break

if fin is None:
    fin = len(paragraphs)

# -----
# c. NO eliminar párrafos intermedios
# Solo determinar posición de inserción
# -----

# Insertaremos justo después del título
pass

# -----
# d. Insertar nuevo contenido debajo del título
# -----

titulo_parrafo = paragraphs[inicio]

if tipo == "tabla":

    tabla = doc.add_table(rows=df.shape[0] + 1, cols=df.shape[1])
    tabla.style = "Table Grid"
    formatear_tabla_word(tabla)

    for col_idx, col in enumerate(df.columns):
        tabla.rows[0].cells[col_idx].text = str(col)

    for row_idx in range(df.shape[0]):
        for col_idx in range(df.shape[1]):

```

```

        tabla.rows[row_idx + 1].cells[col_idx].text =
str(df.iloc[row_idx, col_idx])

    titulo_parrafo._element.addnext(tabla._element)

elif tipo == "imagen":

    nuevo_p = doc.add_paragraph()
    run = nuevo_p.add_run()
    run.add_picture(ruta_imagen, width=Inches(5.5))

    titulo_parrafo._element.addnext(nuevo_p._element)

# 7. GENERADOR DE REPORTES

from docx.opc.exceptions import PackageNotFoundError
from docx.shared import Inches
from docx import Document
from zipfile import BadZipFile

def insertar_tabla_word(doc, df, titulo):

    doc.add_heading(titulo, level=2)

    tabla = doc.add_table(rows=df.shape[0] + 1, cols=df.shape[1])
    tabla.style = "Table Grid"

    # Encabezados
    for col_idx, col in enumerate(df.columns):
        tabla.rows[0].cells[col_idx].text = str(col)

    # Datos
    for row_idx in range(df.shape[0]):
        for col_idx in range(df.shape[1]):
            tabla.rows[row_idx + 1].cells[col_idx].text =
str(df.iloc[row_idx, col_idx])

def generar_sistema_reportes(municipio=None):

    df = pd.read_excel(BASE_EXCEL, sheet_name='CRUDOS')
    df = normalizar_columnas(df)
    df = df[df['RAD_AÑO'].isin(ANIOS)]

```

```

municipios = [municipio] if municipio else df['NOM_MUN'].unique()

os.makedirs(CARPETA_RAIZ, exist_ok=True)

for mun in municipios:

    df_mun = df[df['NOM_MUN'] == mun].copy()
    if df_mun.empty:
        continue

    base_mun = f"{CARPETA_RAIZ}/{mun}"
    os.makedirs(f"{base_mun}/01_Word", exist_ok=True)
    os.makedirs(f"{base_mun}/02_Graficas", exist_ok=True)

    # -----
    # Construcción de tablas
    # -----
    tabla1 = construir_tabla1(df_mun)
    tabla2 = construir_tabla2(df_mun)
    tabla3 = construir_tabla3(df_mun)

    # -----
    # Generación de gráficas
    # -----
    ruta_mes = f"{base_mun}/02_Graficas/Radicados_por_mes.png"
    ruta_dia = f"{base_mun}/02_Graficas/Radicados_por_dia.png"

    graficar_mes(df_mun, ruta_mes)
    graficar_dia(df_mun, ruta_dia)

    # -----
    # Crear o cargar plantilla
    # -----
    try:
        if not os.path.exists(PLANTILLA_WORD):
            raise FileNotFoundError

        doc = Document(PLANTILLA_WORD)

    except (PackageNotFoundError, FileNotFoundError):
        print("⚠️ Plantilla no encontrada. Se generará documento nuevo.")
        doc = Document()

```

```
# Reemplazar municipio en texto
for p in doc.paragraphs:
    if "MPIO" in p.text:
        p.text = p.text.replace("MPIO", mun.upper())

# ORDEN CORRECTO DE INSERCIÓN

reemplazar_bloque(
    doc,
    "Clasificación de los radicados para trámites de conservación
catastral",
    tipo="tabla",
    df=tabla1
)

reemplazar_bloque(
    doc,
    "Radicados por tipo y quien resuelve",
    tipo="tabla",
    df=tabla2
)

reemplazar_bloque(
    doc,
    "Radicados por mes",
    tipo="imagen",
    ruta_imagen=ruta_mes
)

reemplazar_bloque(
    doc,
    "Radicados por día de la semana",
    tipo="imagen",
    ruta_imagen=ruta_dia
)

reemplazar_bloque(
    doc,
    "Radicados resueltos desagregados por quien resuelve y por año
de resolución",
    tipo="tabla",
    df=tabla3
)
```

```
# -----  
# Guardar documento  
# -----  
word_path =  
f"{base_mun}/01_Word/{mun}_Informe_Final_ACC_2024_2025.docx"  
doc.save(word_path)  
  
print(f"✅ Reporte completo generado para {mun}")  
  
# 8. EJECUCIÓN  
  
# Municipio específico  
#generar_sistema_reportes("GUATAVITA")  
  
# Todos los municipios  
generar_sistema_reportes()
```